Unit Testing

Zephyr RTOS Course





Quick Recap



- **Shell Subsystem** — interactive CLI running directly on the embedded device
- Shell backends: UART (default), USB CDC ACM, Telnet, Segger RTT
- Static commands with `SHELL_CMD_REGISTER` and `SHELL_STATIC_SUBCMD_SET_CREATE`
- Dynamic commands with `SHELL_DYNAMIC_CMD_CREATE` for runtime-generated subcommand lists
- `shell_print` / `shell_warn` / `shell_error` — never `printf` from a handler

Format



- Lecture with optional **live demo**
- Homework after each session (checked by us)
- Recordings will be shared
- Questions addressed at the end in a **Q&A session**



Today's Agenda



- Why testing? What is a unit test?
- **Ztest** - writing C tests, suites, hooks
- Assertion families: `zassert_*`, `zexpect_*`, `zassume_*`
- **Twister** - building and running tests
- `twister-out/` - what gets produced
- Code coverage
- Homework



Why Testing?

Unit Testing - What and Why

Type	Scope
Unit	One module in isolation
Integration	Two or more components together
System / E2E	Whole device, real I/O

- Catch bugs at the **earliest, cheapest** point in development
- Prevent regressions — a passing suite means nothing broke
- Run on every commit, in CI, with no hardware required



Ztest - Writing Tests



The Module Under Test: `calculator`



- A simple calculator module with a last-result cache
- APIs return 0 on success and negative errno on failure

```
/* app/modules/calculator/include/calculator.h */
void calculator_reset(void);
int  calculator_get_last_result(int *result);
int  calculator_add(int a, int b, int *result);
int  calculator_sub(int a, int b, int *result);
int  calculator_mul(int a, int b, int *result);
int  calculator_div(int a, int b, int *result);
```



The Test Environment: Files You Need




```
tests/calculator/
├── CMakeLists.txt    # build wiring (test sources + module sources)
├── prj.conf          # CONFIG_ZTEST=y, CONFIG_ZTEST_SHUFFLE=y
├── testcase.yaml     # scenarios for Twister
└── src/
    └── test_basic.c  # all suites and tests
```

- `CMakeLists.txt` - links the test sources with `calculator.c`
- `prj.conf` - enables Ztest and any features the test needs
- `testcase.yaml` - tells Twister which scenarios to build
- `src/` - your actual test code



CMake and `prj.conf`




```
cmake_minimum_required(VERSION 3.20.0)
find_package(Zephyr REQUIRED HINTS $ENV{ZEPHYR_BASE})

project(calculator_unit_test)

target_sources(app PRIVATE
    src/test_basic.c
    ${CMAKE_CURRENT_SOURCE_DIR}/../../app/modules/calculator/src/calculator.c
)
target_include_directories(app PRIVATE
    ${CMAKE_CURRENT_SOURCE_DIR}/../../app/modules/calculator/include
)
```

```
CONFIG_ZTEST=y
CONFIG_ZTEST_SHUFFLE=y
```



`ZTEST_SUITE` - Six Arguments

Zephyr®

```
ZTEST_SUITE(suite_name, predicate, setup, before, after, teardown);
```

Arg	Purpose	When it runs
`suite_name`	Identifier for the suite	-
`predicate`	Returns `true` if suite should run	Once, before suite
`setup`	Allocates / returns a fixture	Once, before first test
`before`	Per-test preparation	Before each test
`after`	Per-test cleanup	After each test
`teardown`	Frees the fixture	Once, after last test



A Minimal Suite




```c
#include <zephyr/ztest.h>
#include "calculator.h"

ZTEST_SUITE(calculator_add, NULL, NULL, NULL, NULL, NULL);

ZTEST(calculator_add, test_positive_numbers)
{
    int result;

    zassert_ok(calculator_add(2, 3, &result), "Addition should succeed");
    zassert_equal(result, 5, "2 + 3 should equal 5");
}
```

- `ZTEST_SUITE` - registers a test suite
- `ZTEST(suite, name)` - registers a test within that suite



The `before()` Hook




```c
static void before(void *f)
{
    ARG_UNUSED(f);
    calculator_reset();  /* reset last-result cache before every test */
}

ZTEST_SUITE(calculator_last_result, NULL, NULL, before, NULL, NULL);
ZTEST_SUITE(calculator_add,         NULL, NULL, before, NULL, NULL);
ZTEST_SUITE(calculator_edge_cases,  NULL, NULL, before, NULL, NULL);
/* ... all six suites share the same hook */
```

- Runs **before every test** in the suite
- The right place to reset shared state

Multiple Suites in One File

```
ZTEST_SUITE(calculator_add, NULL, NULL, before, NULL, NULL);
ZTEST(calculator_add, test_positive_numbers) { /* ... */ }
ZTEST(calculator_add, test_negative_numbers) { /* ... */ }

ZTEST_SUITE(calculator_div, NULL, NULL, before, NULL, NULL);
ZTEST(calculator_div, test_exact_division)   { /* ... */ }
ZTEST(calculator_div, test_divide_by_zero)   { /* ... */ }
ZTEST(calculator_div, test_negative_dividend_truncation) { /* ... */ }
```

- Group tests by **what they exercise** - one suite per public function works well
- Suites can also live in different files - Ztest collects all `ZTEST_SUITE`s linked into the binary



Assertion Families



Three Families, One Job

Zephyr®

Family	On failing condition	Use for
`zassert_*`	Test **stops immediately**	Dependent steps - stop on first failure
`zexpect_*`	Failure recorded, **continues**	Independent checks - report all at once
`zassume_*`	Test **skipped**	Preconditions - "this isn't my test's job"

Decision guide:

- `zassert_*` - "if this fails, nothing after it makes sense"
- `zexpect_*` - "I want all failures visible, not just the first"
- `zassume_*` - "this is a setup step, not the thing I am verifying"

All three share the same suffixes: `_ok`, `_equal`, `_true`, `_not_null`, etc.



Assertion Suffix Reference



Below table covers the most common suffixes. The full API is in the Zephyr Ztest Assert docs.

Suffix	Condition
`_ok(expr)`	`expr == 0`
`_not_ok(expr)`	`expr != 0`
`_equal(a, b)`	`a == b`
`_not_equal(a, b)`	`a != b`
`_true(cond)`	`cond` is true
`_false(cond)`	`cond` is false
`_not_null(ptr)`	`ptr != NULL`
`_is_null(ptr)`	`ptr == NULL`



`zassert_*` in Practice




```
ZTEST(calculator_add, test_positive_numbers)
{
    int result;

    zassert_ok(calculator_add(2, 3, &result), "Addition should succeed");
    zassert_equal(result, 5, "2 + 3 should equal 5");
}

ZTEST(calculator_div, test_divide_by_zero)
{
    int result = 999;

    zassert_equal(calculator_div(10, 0, &result), -EINVAL, "Division by zero should return -
EINVAL");
    zassert_equal(result, 999, "Result should not be modified on error");
}
```



`zexpect_*` and `zassume_*` Together

Zephyr®

```
ZTEST(calculator_edge_cases, test_chained_operations)
{
    int result;

    zassume_ok(calculator_add(10, 20, &result), "precondition: add must succeed");
    zexpect_equal(result, 30, "Expected 10 + 20 = 30, got %d", result);

    zexpect_ok(calculator_sub(result, 5, &result), "sub should succeed");
    zexpect_equal(result, 25, "Expected 30 - 5 = 25, got %d", result);

    zexpect_ok(calculator_mul(result, 2, &result), "mul should succeed");
    zexpect_equal(result, 50, "Expected 25 * 2 = 50, got %d", result);

    zexpect_ok(calculator_div(result, 10, &result), "div should succeed");
    zexpect_equal(result, 5, "Expected 50 / 10 = 5, got %d", result);
}
```



Twister



What is Twister?



- Zephyr's central **test runner and orchestrator**
- Discovers tests by scanning for `testcase.yaml` / `sample.yaml` files
- Builds each test scenario for each matching platform - in parallel
- Runs the binary on host, simulator, or hardware
- Collects results into JSON, JUnit XML, and optional coverage reports
- Same command line on your laptop and in CI



Twister Pipeline



- **Discovery** - finds `testcase.yaml` / `sample.yaml` files recursively
- **Filtering** - applies platform, architecture, tag, and Kconfig constraints
- **Building** - compiles each scenario for each platform (in parallel)
- **Execution** - runs the binary on host, simulator, or real hardware
- **Evaluating** - the configured *harness* parses output and decides pass / fail
- **Reporting** - writes JSON, JUnit XML, optional coverage data



Test Platforms



Platform	What you get	When to pick it
`native_sim`	Full Zephyr OS as native Linux binary	Default for most unit tests
`unit_testing`	Minimal POSIX, no kernel	Pure utilities (no OS deps)
QEMU / Renode	Emulated hardware	Architecture-specific behavior
Real hardware	The real thing	Final validation, HAL coverage



`testcase.yaml` Anatomy




```yaml
common:
  tags:
    - unit
    - calculator
    - ztest

tests:
  example.unit.calculator:
    platform_allow: native_sim
    timeout: 60
```

- `common:` - shared options for every scenario in this file
- `tests:` - one entry per scenario; the key is the **scenario name**



Multiple Scenarios




```
tests:
  example.unit.calculator:
    platform_allow: native_sim
    timeout: 60

  example.unit.calculator.ordered:
    platform_allow: native_sim
    timeout: 60
    extra_args: CONFIG_ZTEST_SHUFFLE=n   # disable shuffle for demo

  example.unit.calculator.hw:
    platform_allow: nrf54l15dk/nrf54l15/cpuapp
    timeout: 120
```



Running Twister




```
# All scenarios in the directory
west twister -T tests/calculator

# Restrict to one platform
west twister -T tests/calculator -p native_sim

# One specific scenario
west twister -T tests/calculator -s example.unit.calculator.ordered

# Verbose - shows each test pass / fail
west twister -T tests/calculator -v    # -vvv for highest verbosity
```

Hardware Testing

Twister flashes the board, captures stdout, and parses the results

```
west twister -T zephyr/samples/hello_world \
    --device-testing \
    --device-serial /dev/ttyACM0 \
    --device-serial-baud 115200 \
    -p nrf52840dk/nrf52840
```

- `--device-testing` - enables running on real hardware
- `--device-serial` - specifies the serial port to capture test output from
- `--device-serial-baud` - configures the baud rate for serial capture



`twister-out/` - What Gets Produced




```
twister-out/
├── twister.json                # CI summary
├── twister.xml                 # JUnit XML
├── twister.log                 # top-level log
├── coverage/                   # if --coverage
│   └── index.html
└── native_sim/<scenario>/
    ├── build.log
    ├── handler.log             # test stdout
    └── zephyr/
        └── zephyr.exe          # native executable — run or debug directly
```



Code Coverage




```
west twister -T tests/calculator \
    -p native_sim \
    --coverage \
    --coverage-tool gcovr \
    --coverage-basedir app/modules/calculator
```

- Output: `twister-out/coverage/index.html` and `twister-out/coverage.json`
- `--coverage-basedir` keeps the report focused on **your** code

What coverage tells you: which source lines and branches were reached

What it does *not* tell you: whether those lines were *correctly* tested



Homework

Homework — Task 1: Implement the Tests

Module under test: : `ring_buf` — circular FIFO buffer. Tag `l8-task1`.

Fill in 7 stub `ZTEST` bodies in `tests/ring_buf/src/test_ring_buf.c`:

- `ring_buf_init`: 1 test — reinit clears state
- `ring_buf_push_pop`: 3 tests — single push/pop, FIFO order, full buffer push
- `ring_buf_boundaries`: 3 tests — peek non-consuming, `pop(NULL)`, `is_full` after fill

```
west twister -T tests/ring_buf -p native_sim
```

Full specification: `tests/ring_buf/TEST_SPEC.md`
Full instructions: `tests/ring_buf/HOMEWORK.md`



Homework — Task 2: Coverage Report



Run the coverage report focused on the `ring_buf` module. Tag `l8-task2`.

```
west twister -T tests/ring_buf -p native_sim \
    --coverage --coverage-tool gcovr \
    --coverage-basedir app/modules/ring_buf
```

1. Open `twister-out/coverage/index.html` and analyze it
2. Include the `twister-out/coverage/` directory in the commit

Full instructions: `tests/ring_buf/HOMEWORK.md`

Thank You

Questions?





Resources



1. Zephyr Ztest Documentation
2. Zephyr Twister Documentation
3. `zassert_*` API reference
4. `zexpect_*` API reference
5. `zassume_*` API reference
6. Getting Started with Twister (Iomico)